

GuruBramha Platform - Database Documentation

1. Database Overview

Database Name: GuruBramha (Configurable via `MONGODB\_URI`)

Purpose of the database: To securely manage and persist all platform data including user identities, elite course content, practice arena challenges, interview preparation roadmaps, transaction histories, and verified DigiLocker documents.

Technologies used: MongoDB (NoSQL Database), Mongoose (Node.js ODM)

Connection structure: The application connects to a MongoDB cluster using a secure connection string defined in the environment variables. The connection is established in `server/index.js` using `mongoose.connect()`

2. Complete Collections Structure

2.1 Users Collection (`users`)

Stores user identities, academic profiles, external portfolio links, and subscription details.

*** **Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key, Auto-generated)
- * `firebaseUid`: String (Required, Unique, Indexed)
- * `displayName`: String (Required)
- * `email`: String (Required, Unique, Indexed)
- * `photoURL`, `phone`, `gender`, `college`, `branch`, `academicYear`, `district`, `state`, `postalCode`: String (Optional)
- * `age`: Number (Optional)
- * `githubUrl`, `leetcodeUrl`, `codechefUrl`: String (Optional)
- * `isSubscribed`: Boolean (Default: `false`)
- * `subscriptionEndDate`: Date (Optional)
- * `scholarPoints`: Number (Default: `0`)
- * `currentStreak`: Number (Default: `0`)
- * `solvedProblems`: Array of ObjectIds (Ref: `Problem`)
- * `bookmarkedCompanies`: Array of ObjectIds (Ref: `Company`)
- * `createdAt`, `updatedAt`: Date (Auto-generated)

2.2 Courses Collection (`courses`)

Stores details for the elite course library and curriculum.

*** **Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key)
- * `title`: String (Required)
- * `instructor`: String (Required)
- * `category`: String (Required)
- * `level`: String (Required, Enum: `Beginner`, `Intermediate`, `Advanced`)
- * `price`: Number (Required)
- * `originalPrice`: Number (Optional)
- * `duration`: String (Optional)
- * `rating`, `reviewsCount`, `studentsEnrolled`: Number (Default: `0`)
- * `thumbnail`, `description`: String (Optional)
- * `lessons`: Array of Objects:
 - * `title` (String, Required)
 - * `duration`, `videoUrl` (String, Optional)
 - * `isFreePreview` (Boolean, Default: `false`)

2.3 Problems Collection (`problems`)

Stores coding challenges for the Elite Practice Arena.

*** **Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key)
- * `title`: String (Required)
- * `difficulty`: String (Required, Enum: `Easy`, `Medium`, `Hard`)
- * `category`: String (Required)
- * `description`: String (Required)
- * `tags`: Array of Strings
- * `accuracy`: Number (Default: `0`)
- * `points`: Number (Default: `100`)
- * `validationCases`: Array of Objects:
 - * `input`, `output` (String)
- * `isHidden` (Boolean, Default: `false`)

2.4 Companies Collection (`companies`)

Stores interview preparation roadmaps and placement protocols.

****Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key)
- * `name`: String (Required)
- * `type`: String (Required, Enum: `Product Based`, `Service Based`, `Startup`)
- * `difficulty`: String (Enum: `Easy`, `Medium`, `Hard`, `Expert`)
- * `logo`, `package`, `overview`, `eligibility`: String (Optional)
- * `skills`, `process`: Array of Strings
- * `rounds`: Array of Objects (`title` String, `topics` Array of Strings)
- * `roadmap30Days`: Array of Objects (`week` Number, `focus` String)

2.5 Transactions Collection (`transactions`)

Stores Razorpay payment and subscription details.

****Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key)
- * `user`: ObjectId (Required, Foreign Key -> `User`)
- * `amount`: Number (Required)
- * `currency`: String (Default: `INR`)
- * `razorpayOrderId`: String (Required, Indexed)
- * `razorpayPaymentId`, `razorpaySignature`, `receipt`: String (Optional)
- * `status`: String (Enum: `created`, `successful`, `failed`, Default: `created`)

2.6 Documents Collection (`documents`)

Stores metadata for certificates fetched via DigiLocker.

****Fields & Data Types**:**

- * `\_id`: ObjectId (Primary Key)
- * `user`: ObjectId (Required, Foreign Key -> `User`)
- * `digilockerId`: String (Required)
- * `name`, `uri`: String (Required)
- * `type`: String (Optional)
- * `issueDate`: Date (Optional)

3. User Side Database Structure

From the user's perspective, the database structure facilitates the following:

* **Authentication/Login**: Managed via Firebase on the frontend, which syncs with the `users` collection using the `firebaseUid`.

* **Profile Data**: Users can update their academic history, age, gender, and social links (GitHub, LeetCode) directly into the `users` table.

* **Courses**: Users browse the `courses` collection. Premium courses check the `user.isSubscribed` flag before granting access to video URLs.

* **Subscriptions & Payments**: Initiating a purchase creates a `transaction` document. Upon success via Razorpay, the `user.isSubscribed` flag is set to `true` and the `subscriptionEndDate` is extended.

* **Practice/Coding**: Users query the `problems` collection. Successful submissions append the Problem's ObjectId to `user.solvedProblems` and increase `user.scholarPoints`.

* **Documents**: Users authenticating with DigiLocker create records in the `documents` collection, linking official certificates to their profile via the `user` foreign key.

4. Admin Side Database Structure

The admin perspective provides full control over the platform's content and tracking:

* **User Management**: Admins can query the `users` collection to monitor `scholarPoints`, view academic demographics, or manually revoke/grant subscriptions (`isSubscribed`).

* **Content Management**: Admins have CRUD access to `courses`, `problems`, and `companies`. They can add new coding challenges, update curriculum videos, and add new interview roadmaps.

* **Transactions & Reports**: Admins query the `transactions` collection filtered by `status: 'successful'` to generate revenue reports and track active subscriptions.

* **Analytics**: Aggregating `user.solvedProblems`, `course.studentsEnrolled`, and `problem.accuracy` helps admins track platform engagement.

5. Flow Charts

User Visits Website



Login / Signup



User Dashboard

↓

Courses / Coding Practice / Interview Preparation

↓

Payment Gateway (Razorpay)

↓

Store Data in MongoDB

↓

Access Premium Features

6. API & Backend Mapping

The frontend communicates with the MongoDB collections via the following expected REST API mapping:

****Authentication / Profile****

* `POST /api/auth/sync` -> Maps to `User.findOneOrCreate()`

* `GET /api/users/profile` -> Maps to `User.findById()`

* `PUT /api/users/profile` -> Maps to `User.findByIdAndUpdate()`

****Courses****

* `GET /api/courses` -> Maps to `Course.find()`

* `GET /api/courses/:id` -> Maps to `Course.findById()` (Protected payload checks `User.isSubscribed`)

****Practice Arena****

* `GET /api/problems` -> Maps to `Problem.find()`

* `POST /api/problems/:id/submit` -> Verifies code, updates `User.solvedProblems` & `User.scholarPoints`

****Interview Roadmaps****

* `GET /api/companies` -> Maps to `Company.find()`

* `POST /api/companies/:id/bookmark` -> Updates `User.bookmarkedCompanies`

****Payments (/api/payment)****

* `POST /order` -> Uses Razorpay API, creates `Transaction` doc

* `POST /verify` -> Verifies signature, updates `Transaction` status, updates `User` subscription